WIA1006/WID3006 MACHINE LEARNING

Semester 2, Session 2025/2026 | Universiti Malaya

NEURAL NETWORK

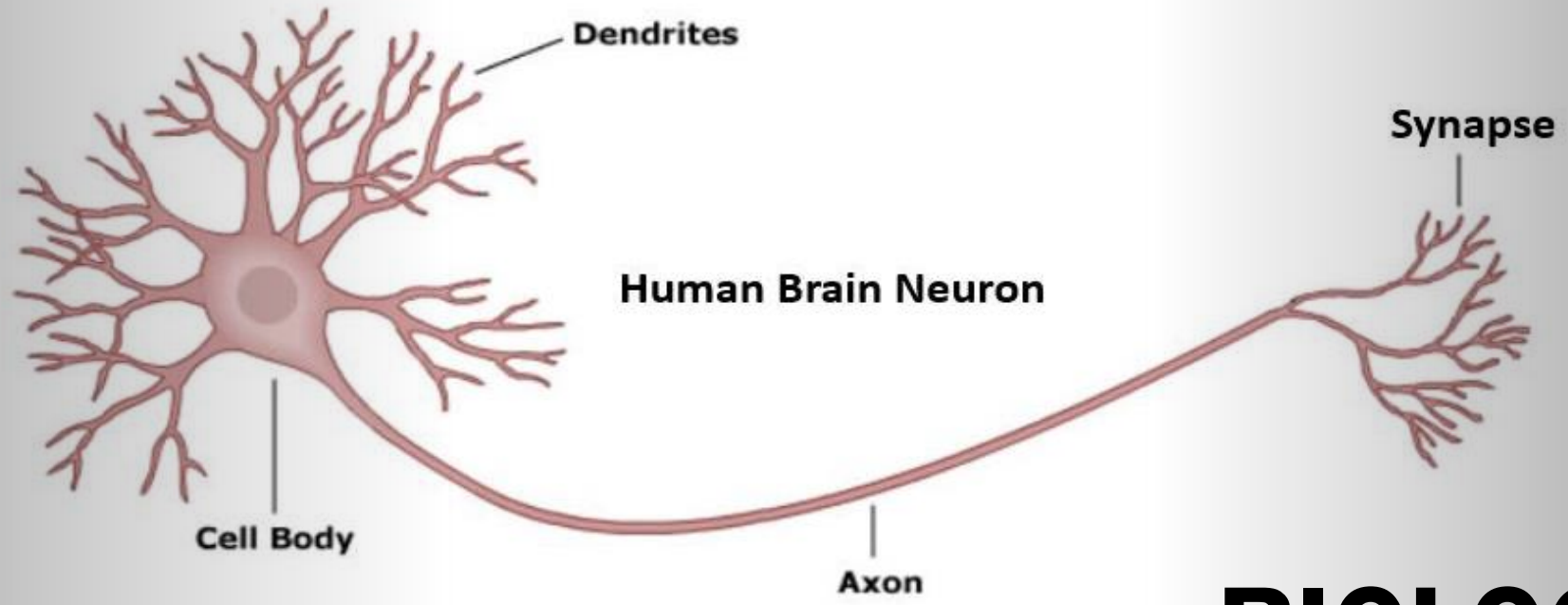
INTRODUCTION

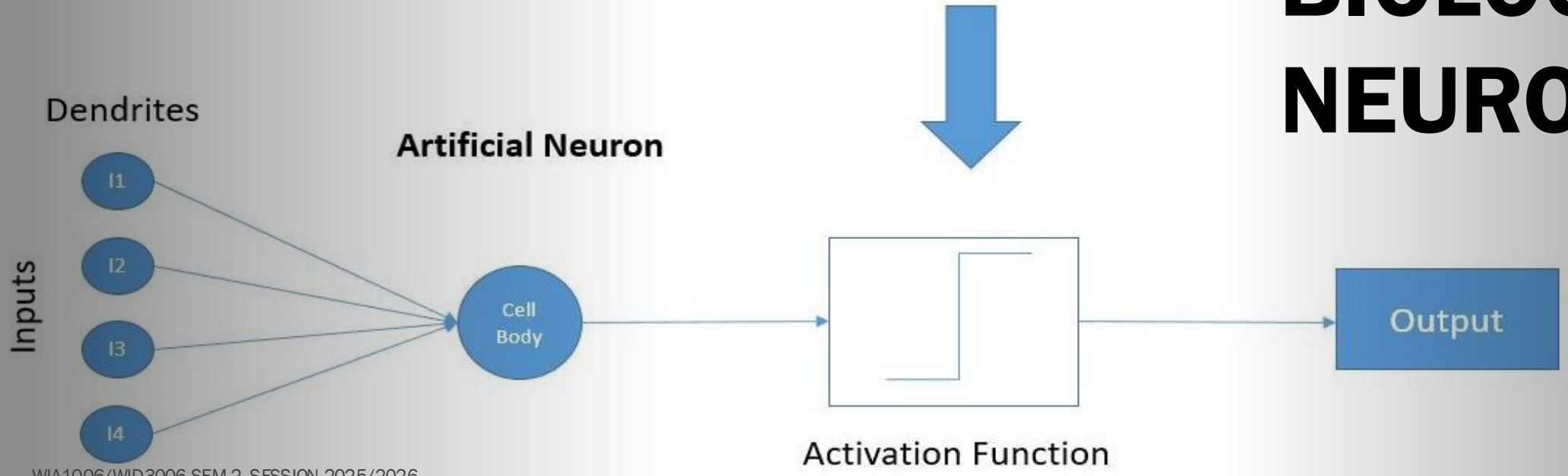
- Why ANN?
  - Some tasks can be done easily (effortlessly) by humans but are hard by conventional paradigms on Von Neumann machine with algorithmic approach
    - Pattern recognition (old friends, hand-written characters)
    - Content addressable recall
    - Approximate, common sense reasoning (driving, playing piano, baseball player)
  - These tasks are often ill-defined, experience based, hard to apply logic

INTRODUCTION

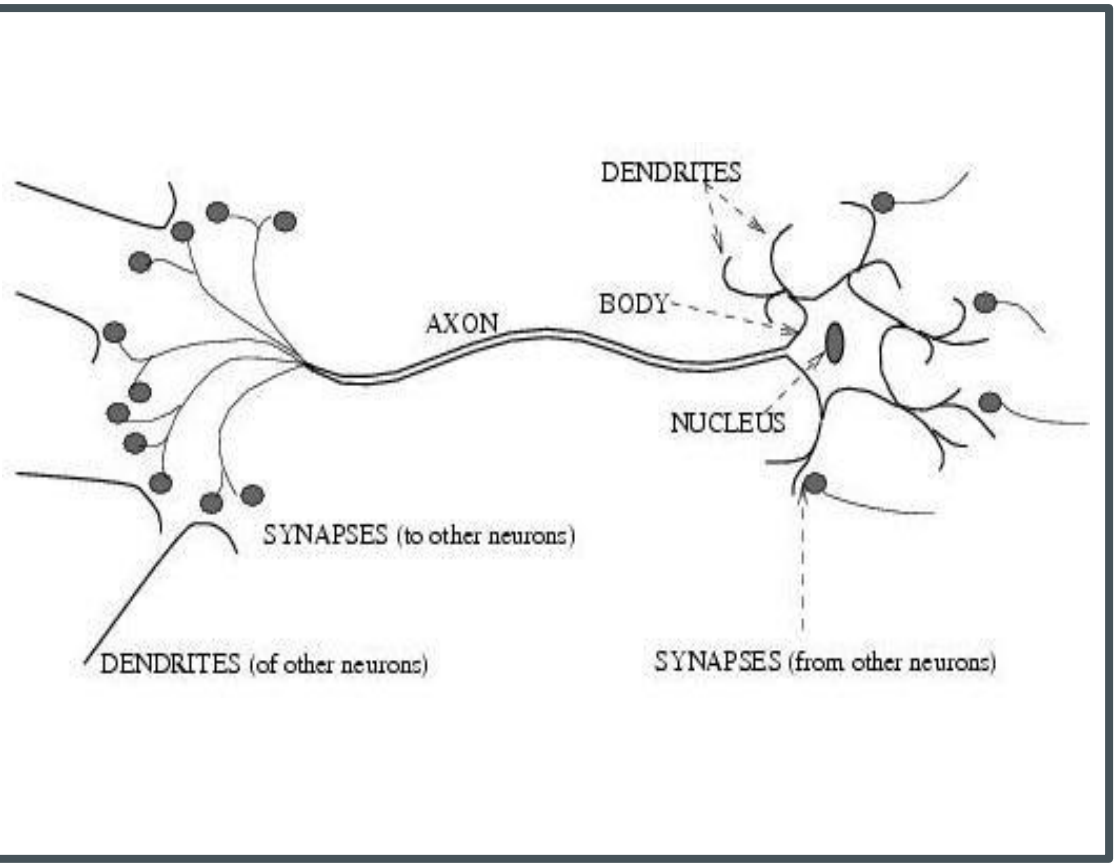
Von Neumann Machine	Human brain
One or a few high speed (ns) processors with considerable computing power	Large # (10^{11}) of low-speed processors (ms) with limited computing power
One or a few shared high-speed buses for communication	Large # (10^{15}) of low-speed connections
Sequential memory access by address	Content addressable recall (CAM)
Problem-solving knowledge is separated from the computing component	Problem-solving knowledge resides in the connectivity of neurons
Hard to be adaptive	Adaptation by changing the connectivity

BIOLOGICAL NEURON





BILOGICAL NEURON ACTIVITY

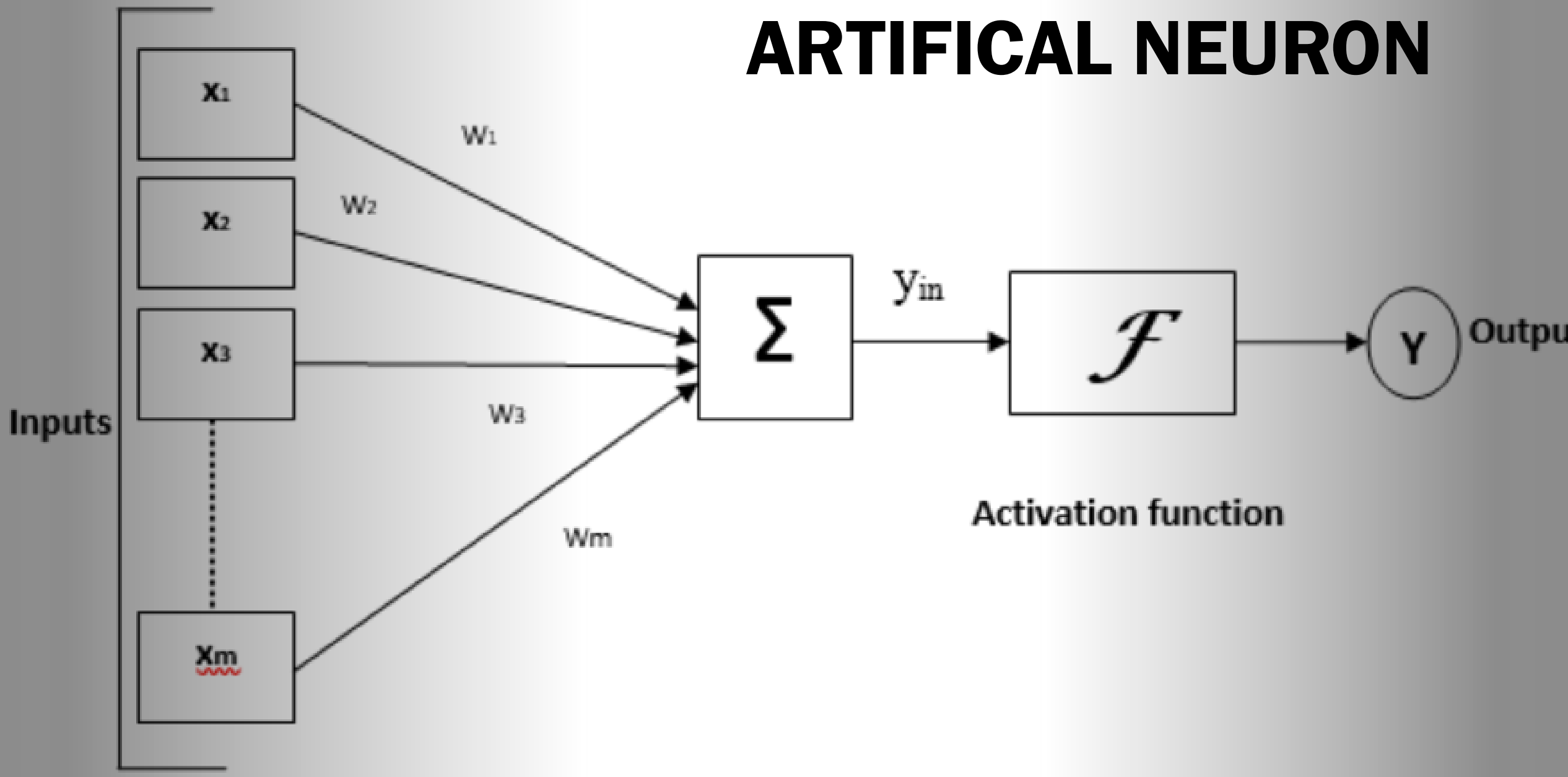


- Each neuron has a *body*, an *axon*, and many *dendrites*
  - Can be in one of the two states: *firing* and *rest.*
  - Neuron fires if the total incoming stimulus exceeds the threshold
- *Synapse*: thin gap between axon of one neuron and dendrite of another.
  - Signal exchange
  - Synaptic strength/efficiency

INTRODUCTION TO ANN

- **What is an (artificial) neural network**
  - A set of **nodes** (units, neurons, processing elements)
    - Each node has input and output
    - Each node performs a simple computation by its **node function**
  - **Weighted connections** between nodes
    - Connectivity gives the structure/architecture of the net
    - What can be computed by a NN is primarily determined by the connections and their weights
  - A very much simplified version of networks of neurons in animal nerve systems

ARTIFICAL NEURON



BNN VS. ANN

BNN	ANN
Massively parallel, slow but superior than ANN	Massively parallel, fast but inferior than BNN
10^{11} neurons and 10^{15} interconnections	10^{2} to 10^{4} nodes; mainly depends on the type of applications and network designer
Tolerate ambiguity	Very precise, structured and formatted data is required to tolerate ambiguity
Performance degrades with even partial damage	It is capable of robust performance, hence has the potential to be fault tolerant
Stores the information in the synapse	Stores the information in continuous memory locations

APPLICATIONS

Neural Network Applications can be grouped in the following categories:

- **Clustering**
  - A clustering algorithm explores the similarity between patterns and places similar patterns in a cluster. Best known applications include data compression and data mining
- **Classification/Pattern recognition**
  - The task of pattern recognition is to assign an input pattern (like handwritten symbol) to one of many classes. This category includes algorithmic implementations such as associative memory.
- **Function approximation**
  - The tasks of function approximation is to find an estimate of the unknown function f() subject to noise. Various engineering and scientific disciplines require function approximation.
- **Prediction/Dynamical systems**
  - The task is to forecast some future values of a time-sequenced data. Prediction has a significant impact on decision support systems. Prediction differs from Function approximation by considering time factor. Here the system is dynamic and may produce different results for the same input data based on system state (time).

TYPES OF NEURAL NETWORK

Neural Network types can be classified based on following attributes:

Applications

- Classification
- Clustering
- Function approximation
- Prediction

Connection Type

- Static (feedforward)
- Dynamic (feedback)

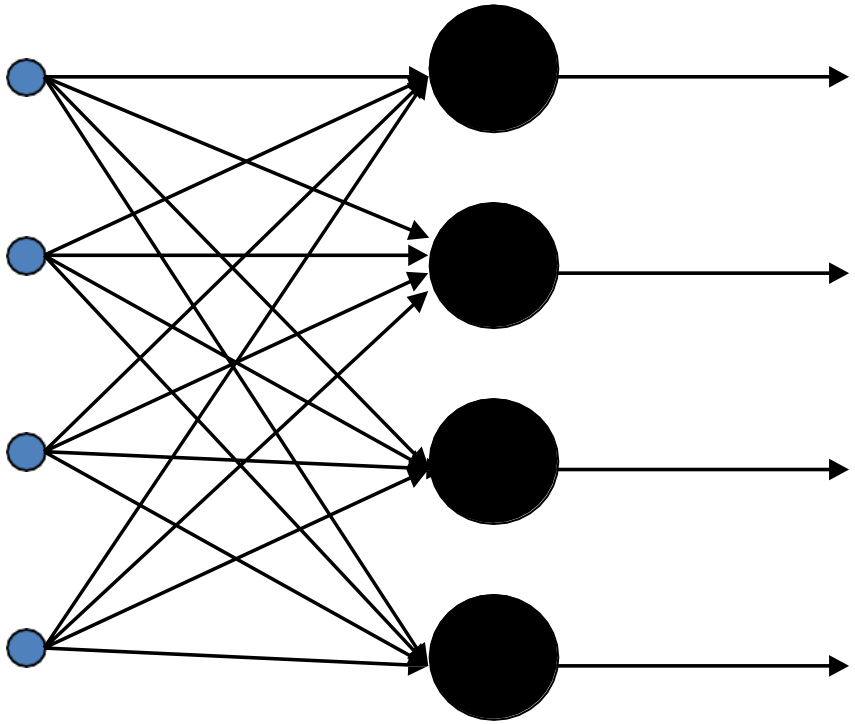
Topology

- Single layer
- Multilayer
- Recurrent
- Self-organized

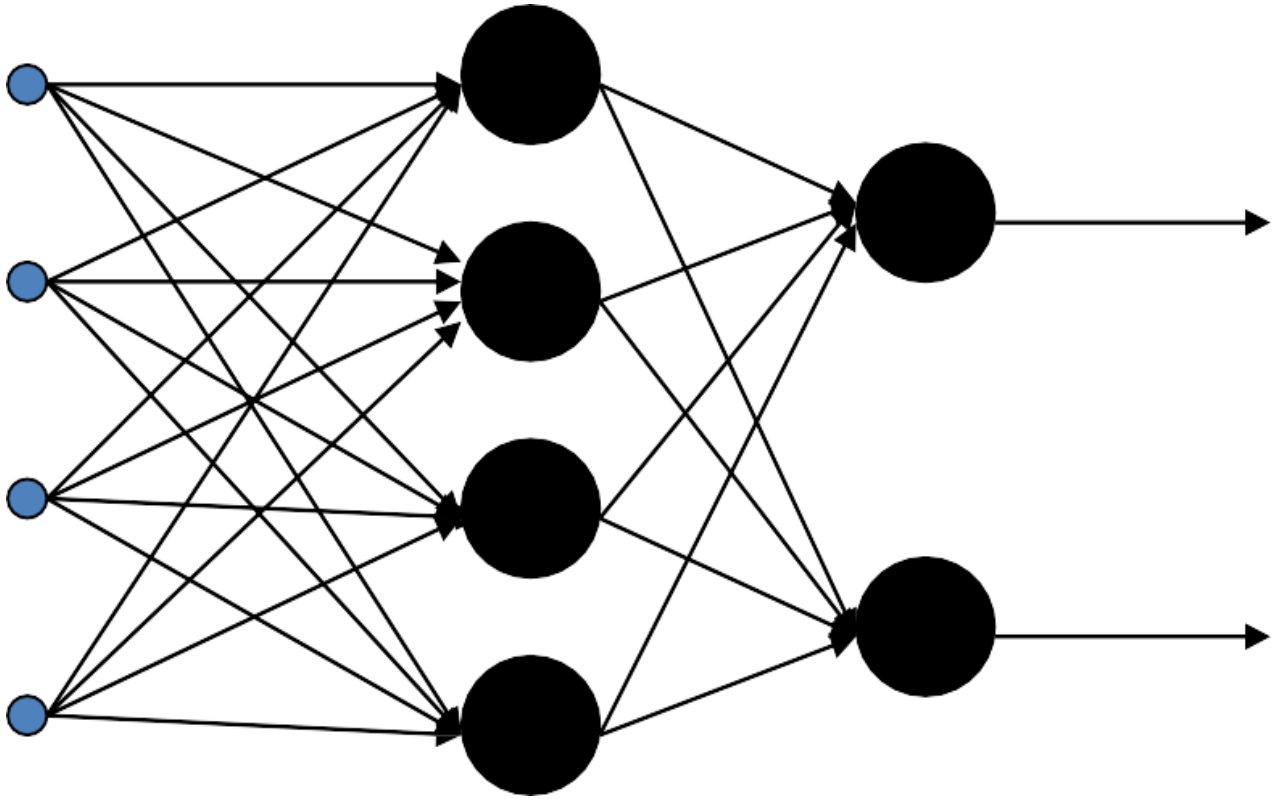
Learning Methods

- Supervised
- Unsupervised

TYPES OF NETWORKS

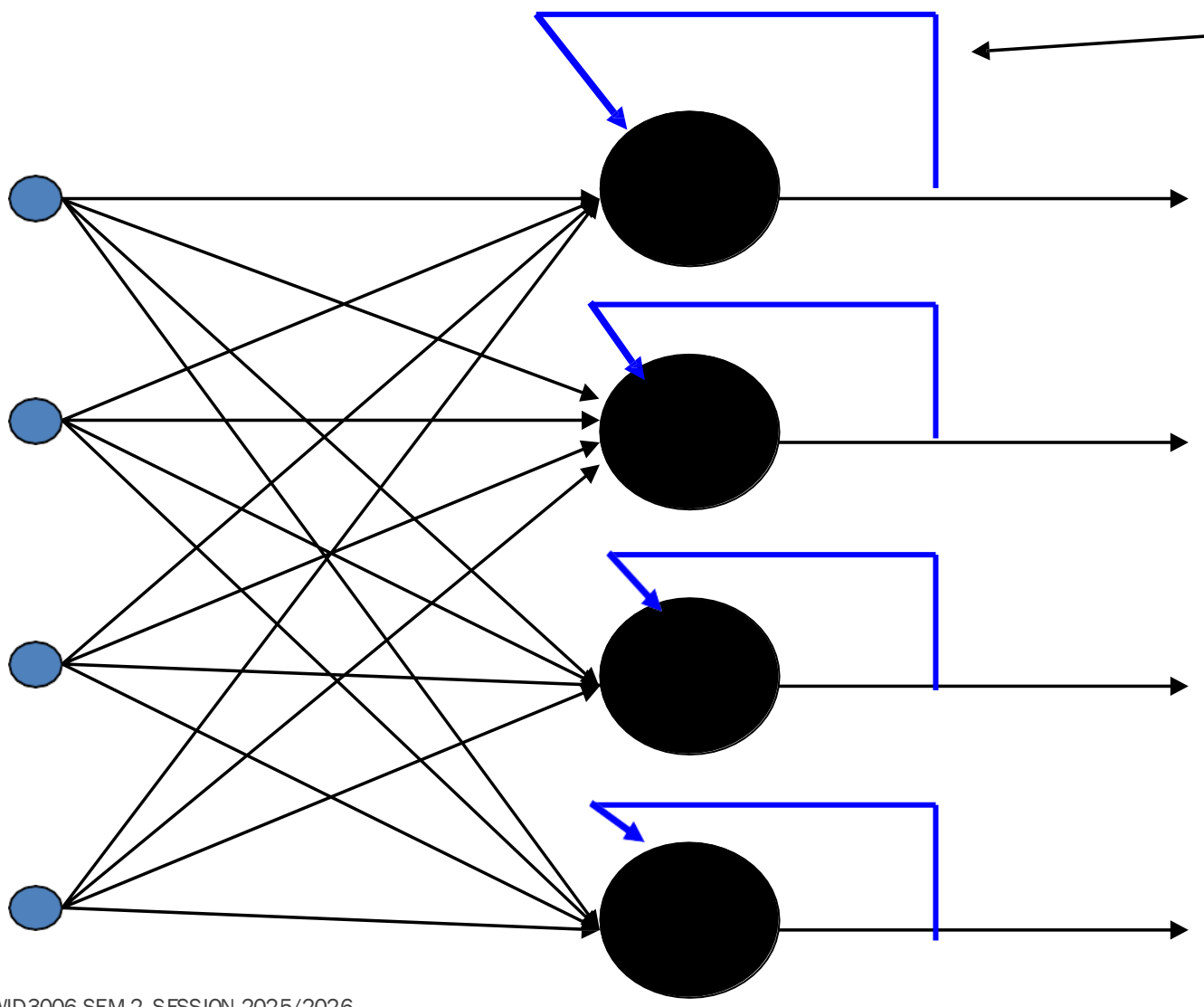


Multiple Inputs and Single Layer



Multiple Inputs and layers

TYPES OF NETWORKS



Recurrent Networks

LEARNING TECHNIQUES

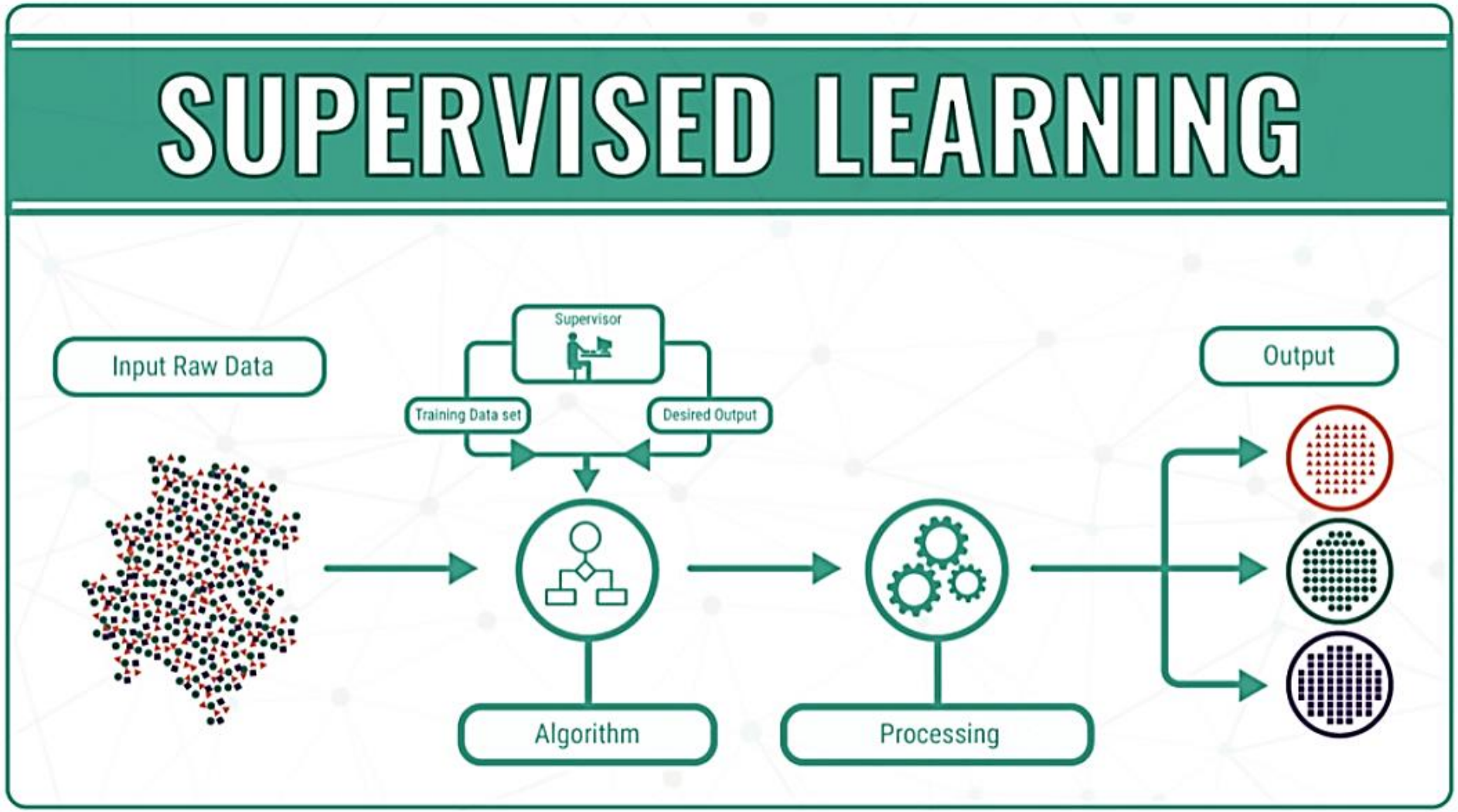


Figure credit to: https://www.linkedin.com/pulse/machine-learning-explained-understanding-supervised-ronald-van-loon/

LEARNING TECHNIQUES

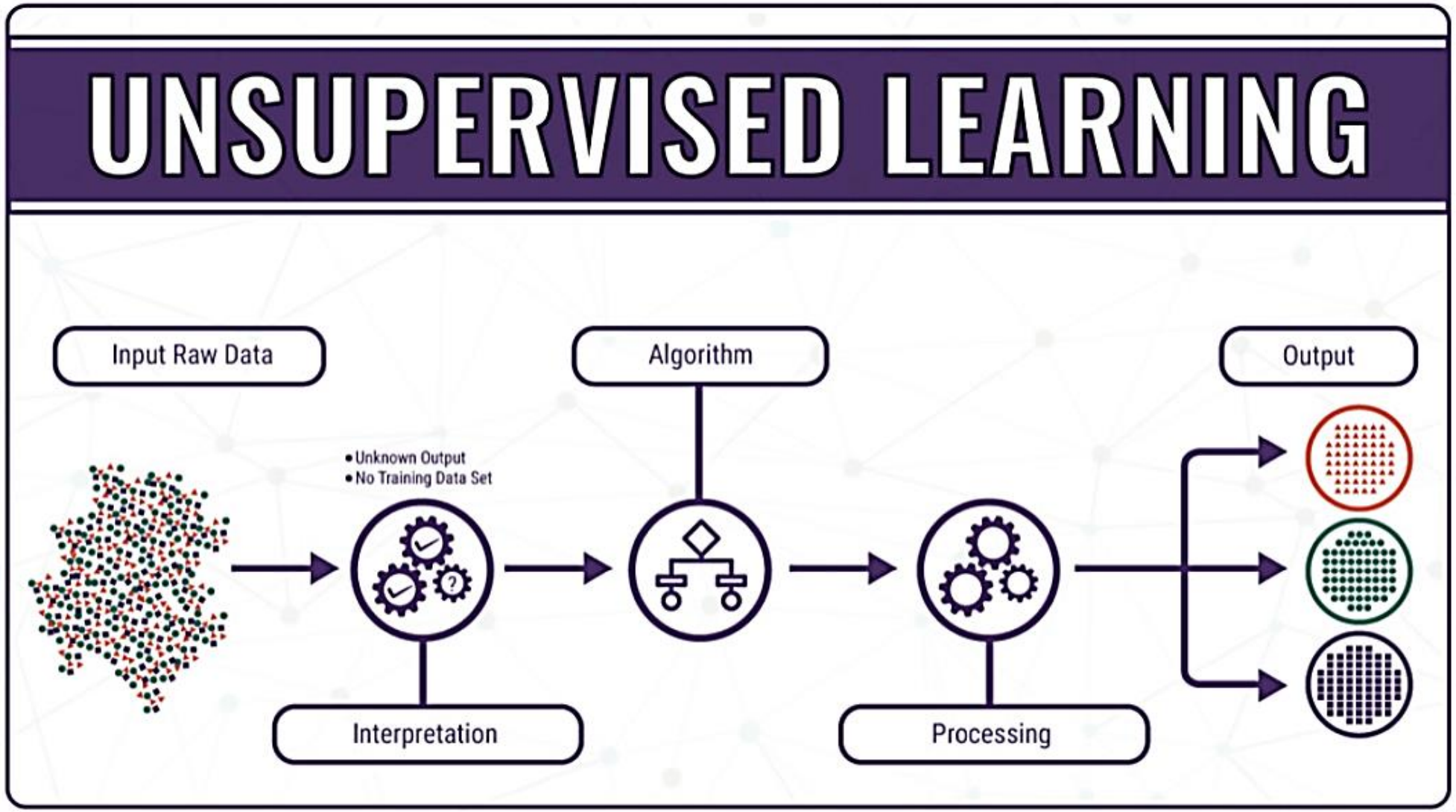


Figure credit to: https://www.linkedin.com/pulse/machine-learning-explained-understanding-supervised-ronald-van-loon/

LEARNING TECHNIQUES

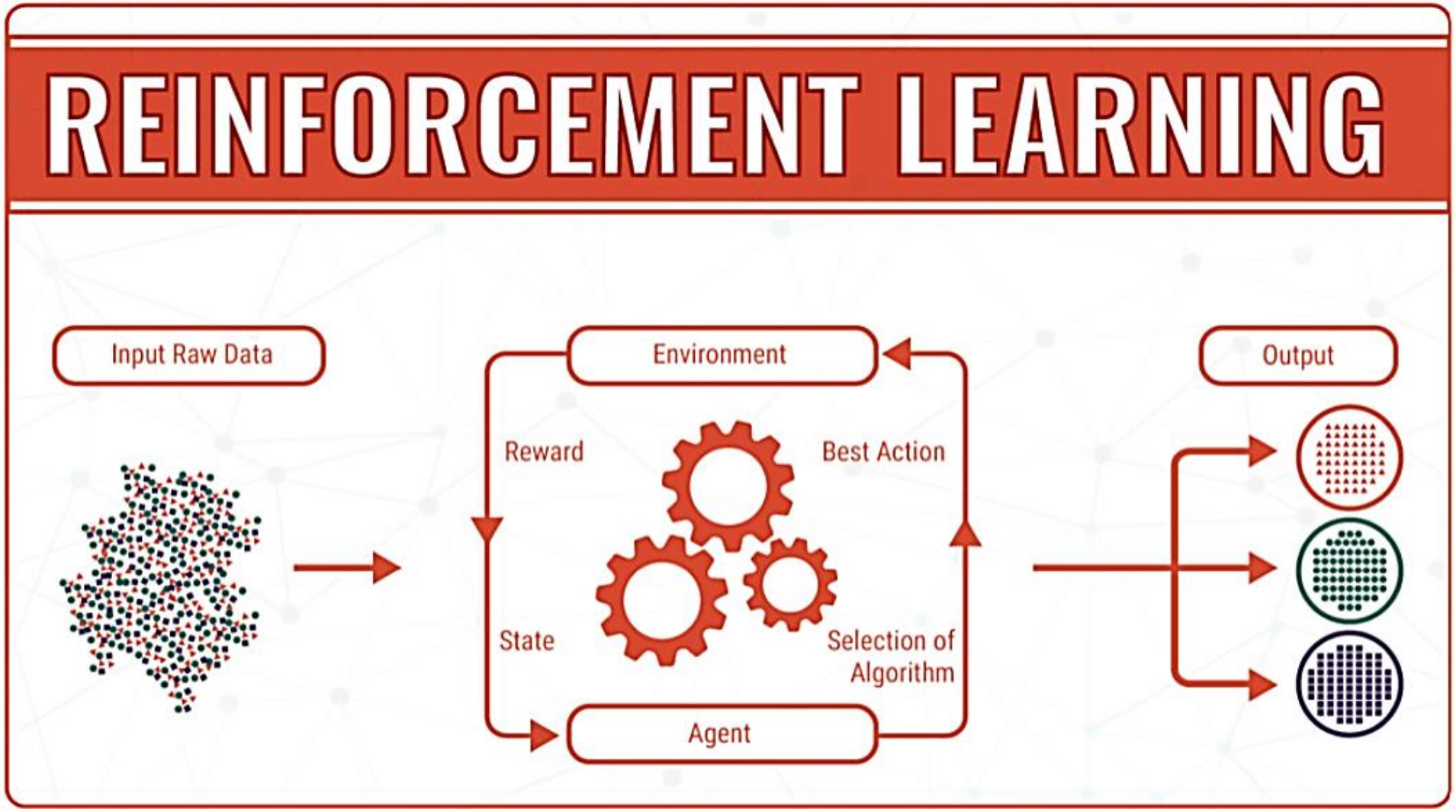
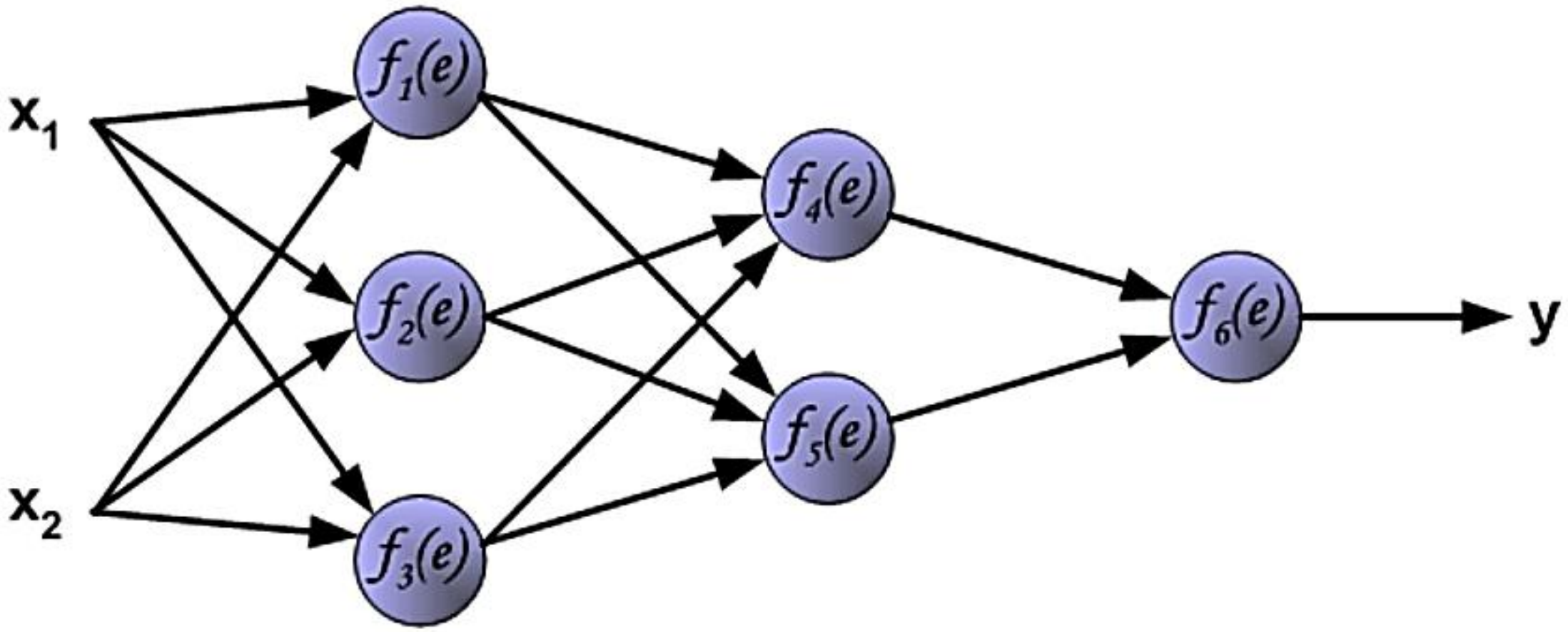
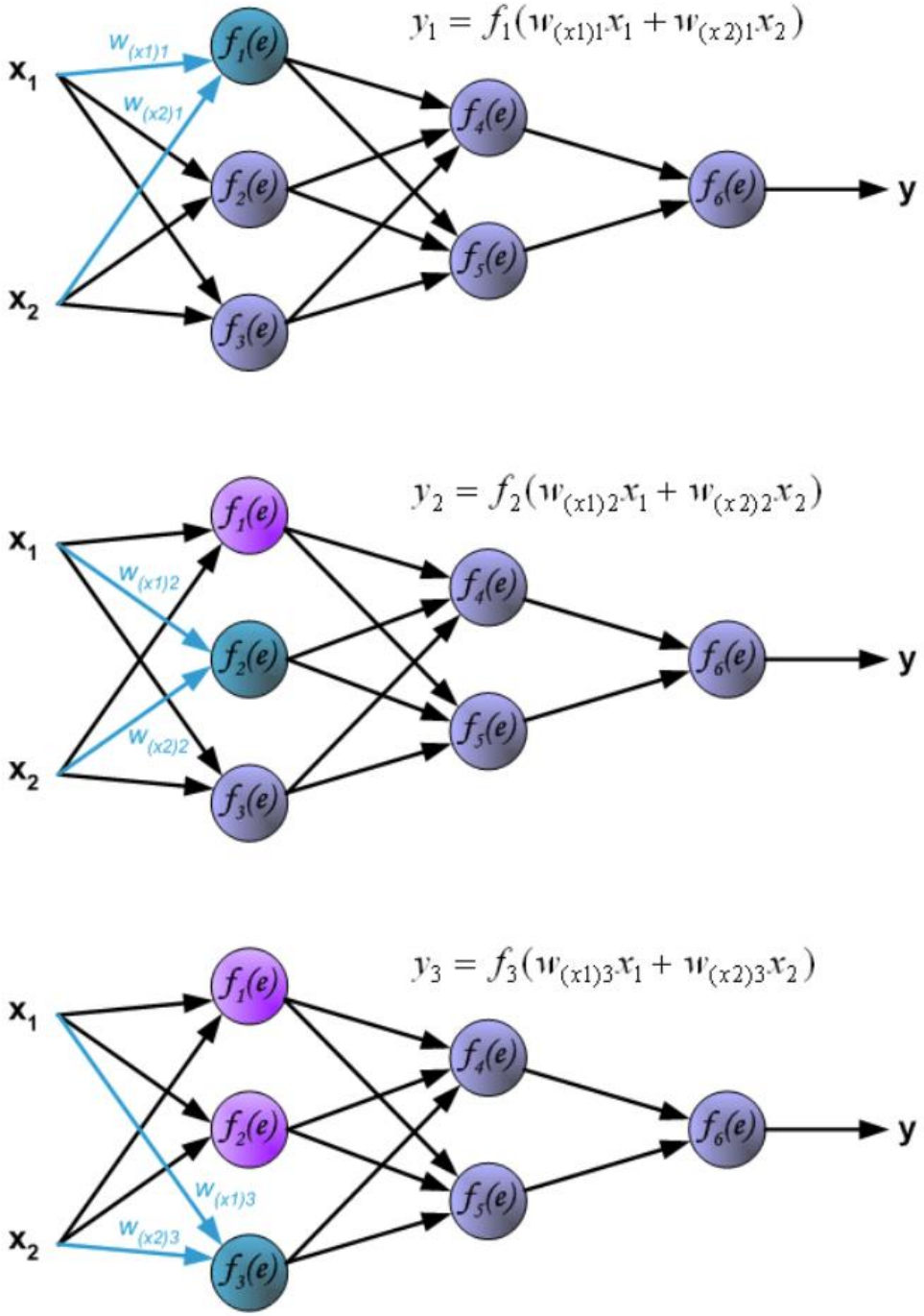


Figure credit to: https://www.linkedin.com/pulse/machine-learning-explained-understanding-supervised-ronald-van-loon/

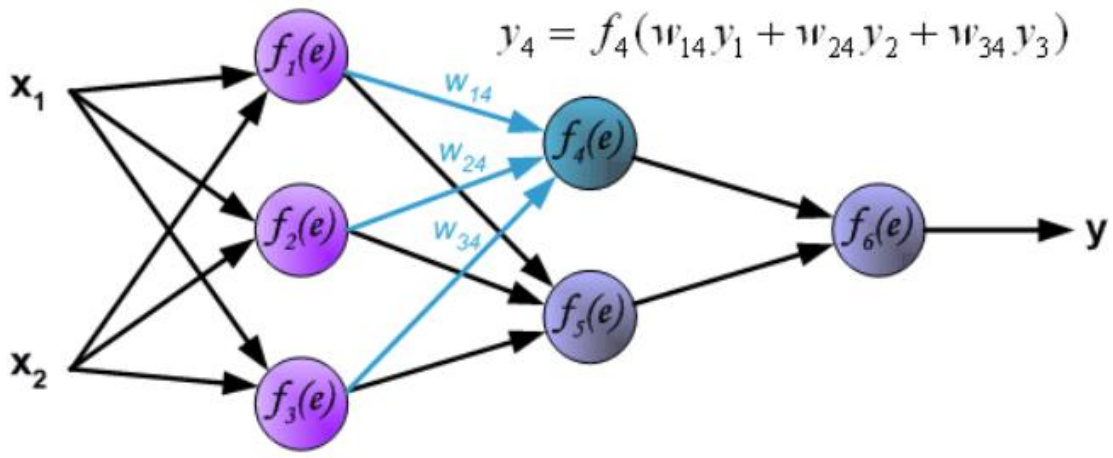
MULTILAYER PERCEPTRON

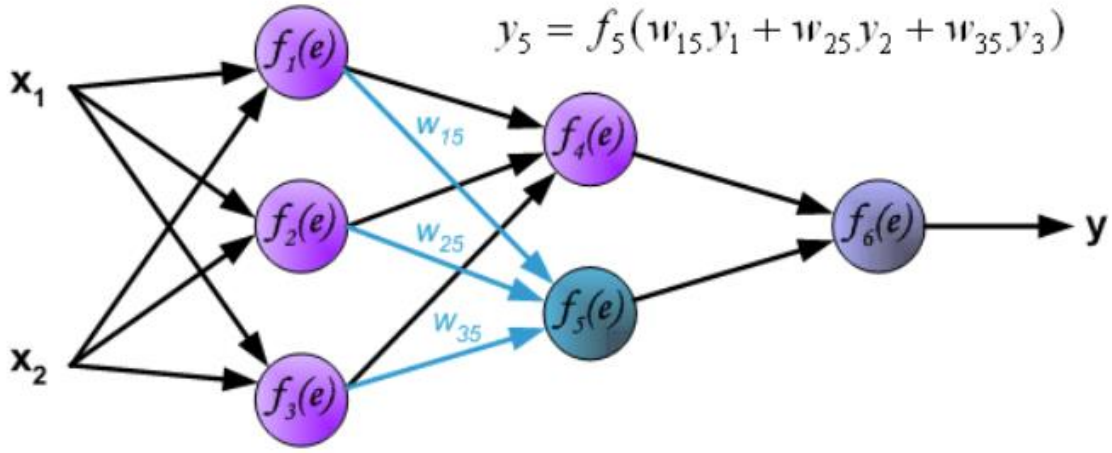


TRAINING MULTILAYER NETWORK USING BACKPROPAGATION

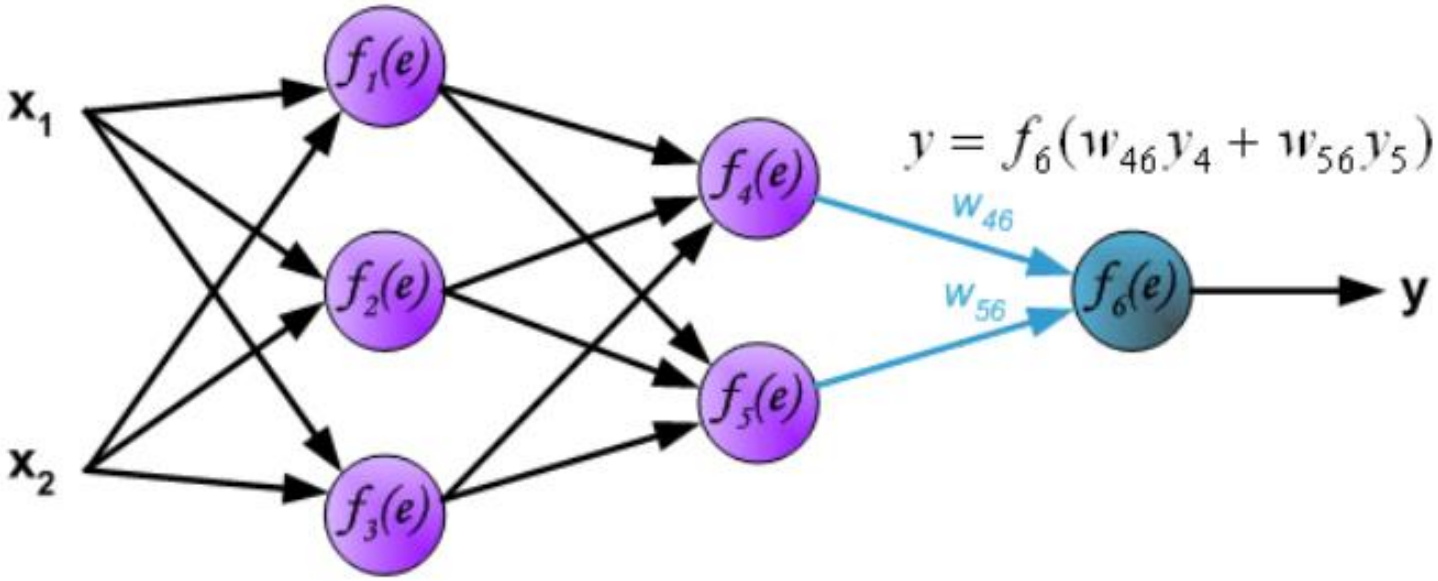


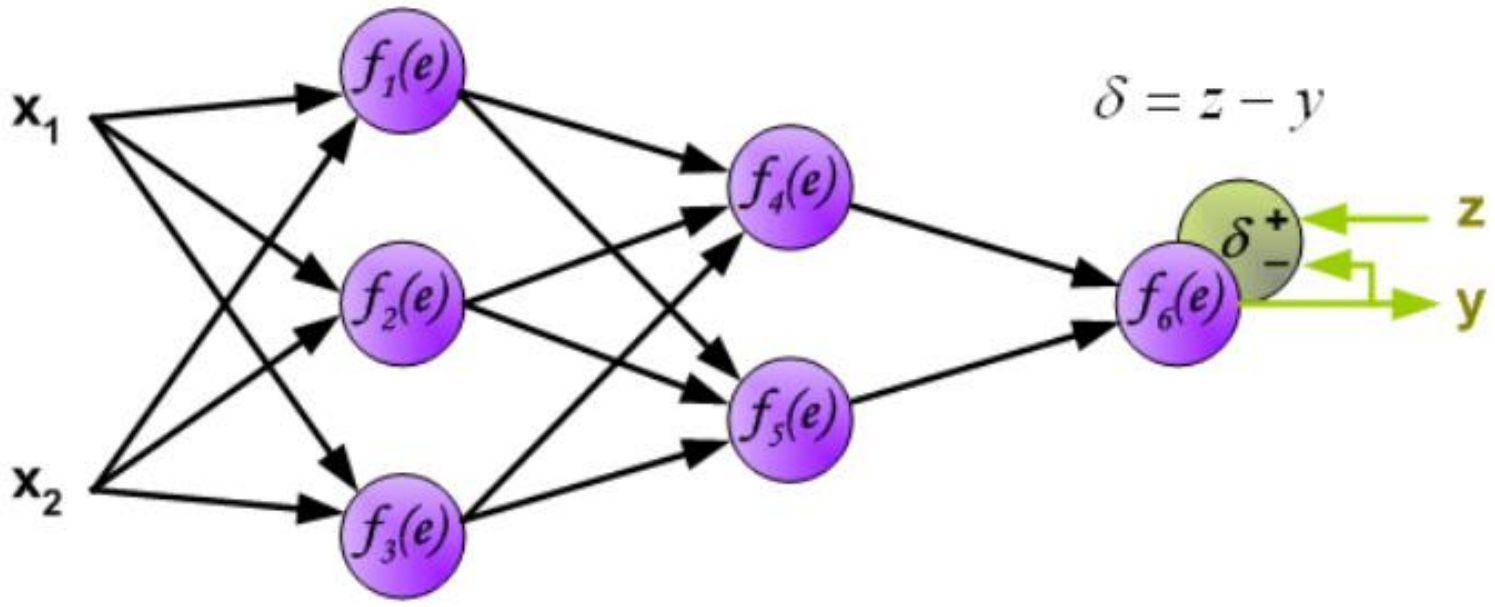
TRAINING MULTILAYER NETWORK USING BACKPROPAGATION





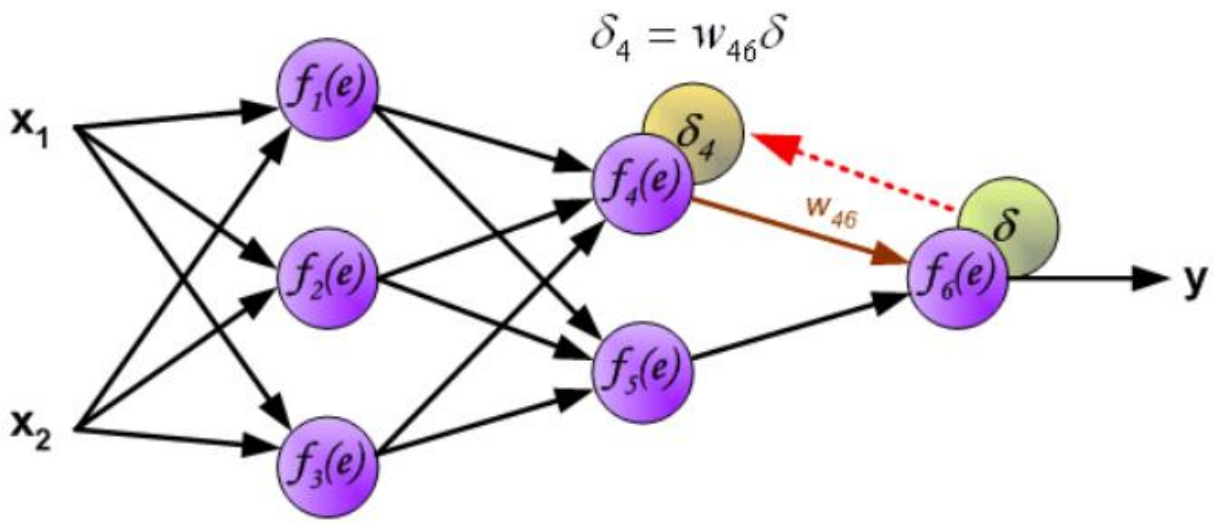
TRAINING MULTILAYER NETWORK USING BACKPROPAGATION

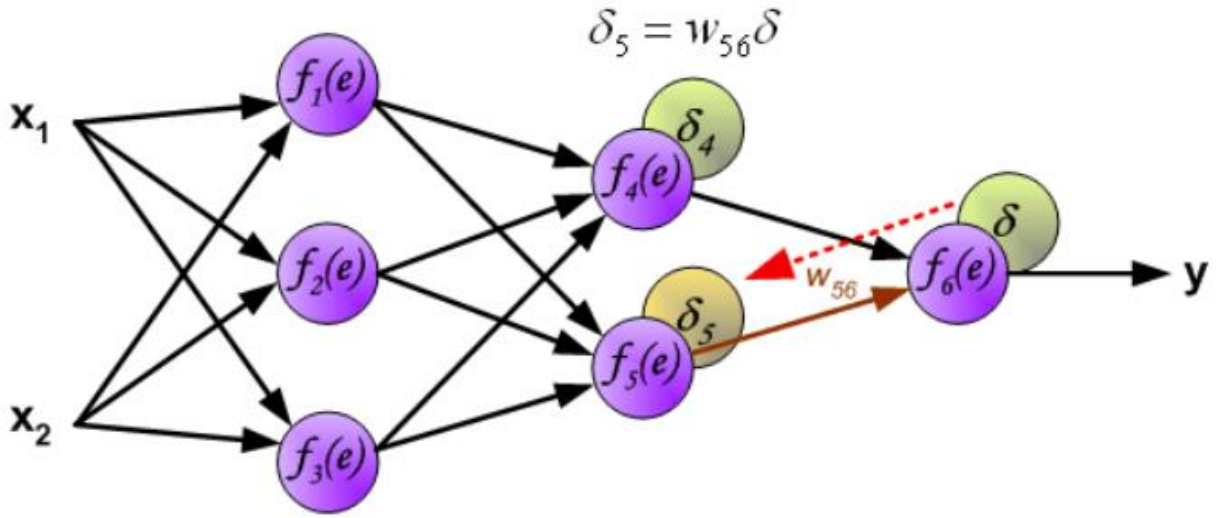




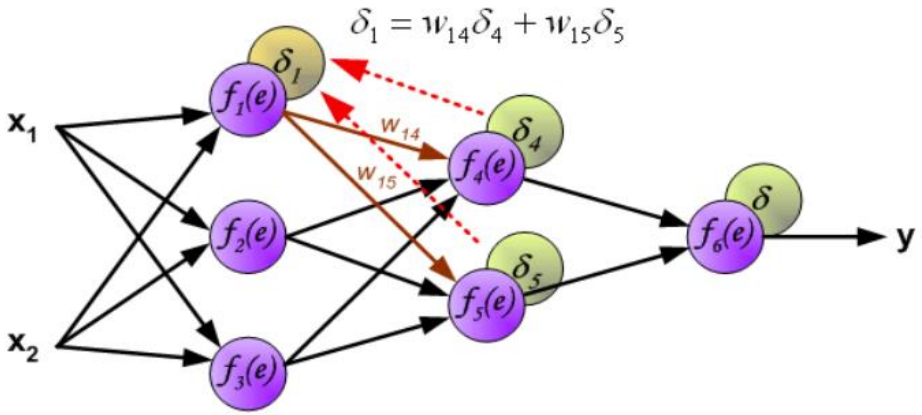
TRAINING MULTILAYER NETWORK USING BACKPROPAGATION

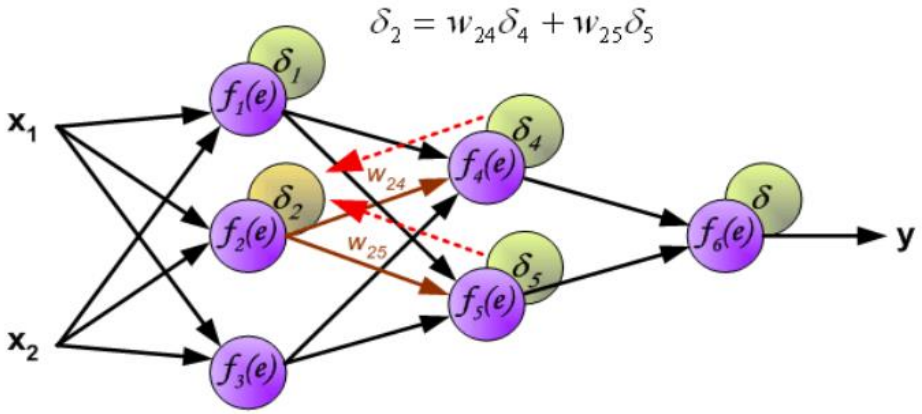
TRAINING MULTILAYER NETWORK USING BACKPROPAGATION

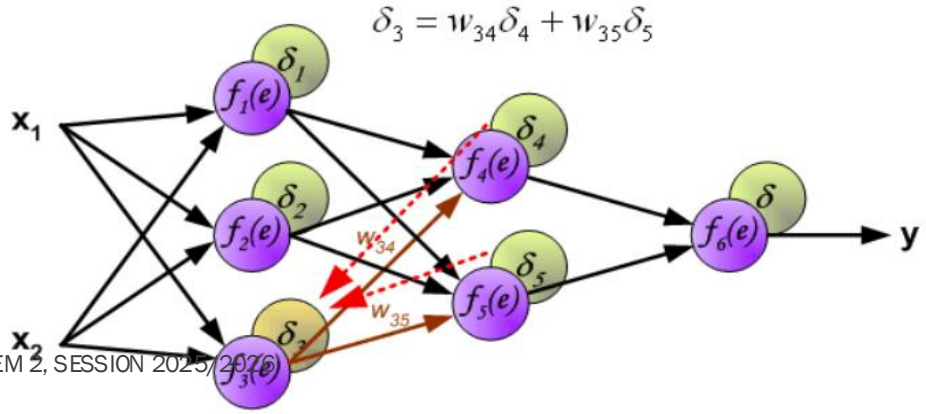




TRAINING MULTILAYER NETWORK USING BACKPROPAGATION



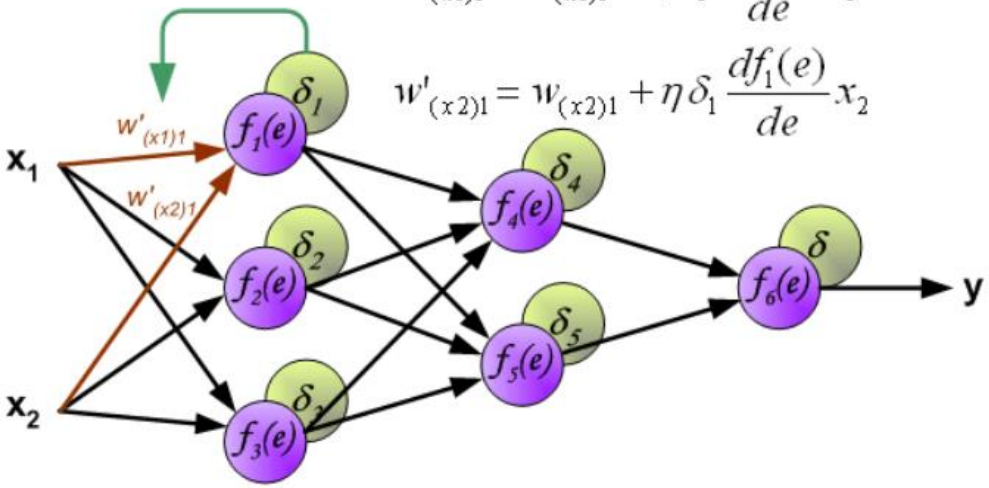




TRAINING MULTILAYER NETWORK USING BACKPROPAGATION

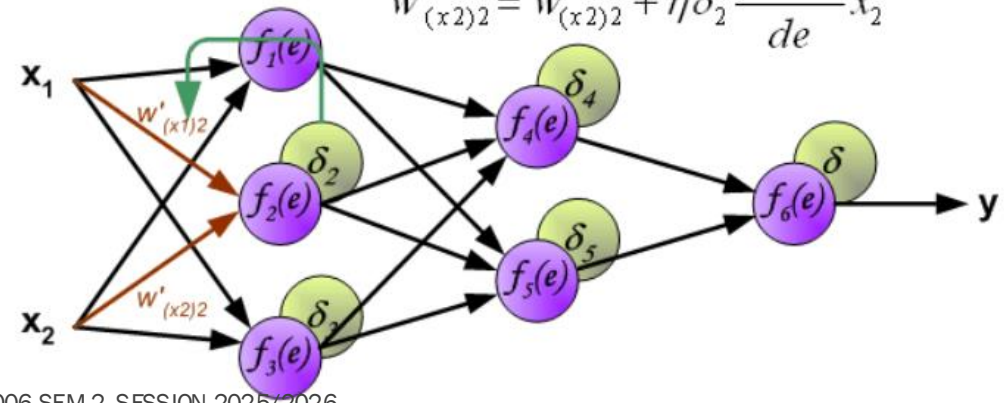
$$w'_{(x1)1} = w_{(x1)1} + \eta \delta_1 \frac{df_1(e)}{de} x_1$$

$$w'_{(x2)1} = w_{(x2)1} + \eta \delta_1 \frac{df_1(e)}{de} x_2$$



$$w'_{(x1)2} = w_{(x1)2} + \eta \delta_2 \frac{df_2(e)}{de} x_1$$

$$w'_{(x2)2} = w_{(x2)2} + \eta \delta_2 \frac{df_2(e)}{de} x_2$$



BACKPROPAGATION NOTATION

- Weights: two weight matrices:
  - $w^{(1,0)}$ from input layer (0) to hidden layer (1)
  - $w^{(2,1)}$ from hidden layer (1) to output layer (2)
  - $w_{2,1}^{(1,0)}$ weight from node 1 at layer 0 to node 2 in layer 1

- Training samples: pair of $\{(x_p, d_p) p = 1, \ldots, P\}$ so it is supervised learning
- Input pattern: $x_p = (x_{p,1}, \ldots, x_{p,n})$
- Output pattern: $y_p = (y_{p,1}, \ldots, y_{p,k})$
- Desired output: $d_p = (d_{p,1}, \ldots, d_{p,k})$
- Error: $l_{p,j} = d_{p,k} - y_{p,k}$ error for output node *j* when x_p is applied

  Sum squared error = $\sum_{p=1}^{P} \sum_{j=1}^{K} l_{p,j}$

  This error drives learning (change $w^{(1,0)}$ and $w^{(2,1)}$)

BACKPROPAGATION LEARNING

- Sigmoid function:

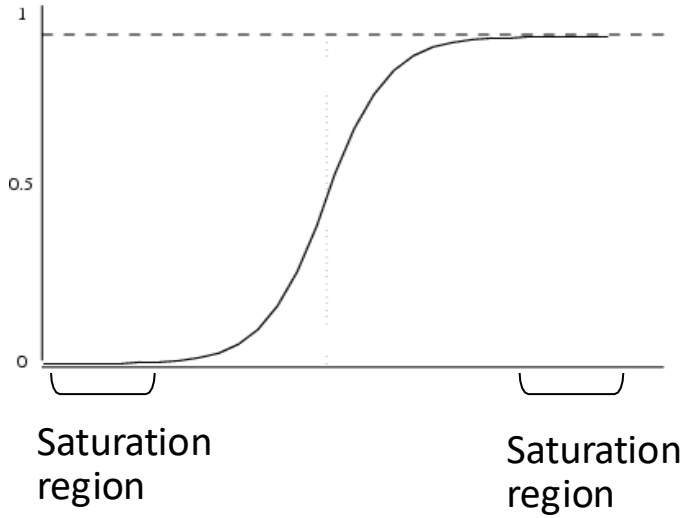
Differentiable: $S(x) = \frac{1}{1+e^{-1}}$

$$S'(x) = \frac{1}{(1+e^{-x})^2} \cdot (1 + e^{-x})'$$

$$= -\frac{1}{(1+e^{-x})^2} \cdot (e^{-x})$$

$$= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}}$$

$$= S(x)(1 - S(x))$$



- When $|net|$ is sufficiently large, it moves into one of the two saturation regions, behaving like a threshold or ramp function.

BACKPROPAGATION LEARNING

- Chain rule of differentiation:

$$\mathrm{z} = \mathrm{f}(\mathrm{x}), \mathrm{y} = \mathrm{g}(\mathrm{x}), \mathrm{x} = \mathrm{h}(\mathrm{t}), \text{then}$$

$$\frac{d(z)}{d(t)} = \frac{d(z)}{d(y)} \cdot \frac{d(y)}{d(x)} \cdot \frac{d(x)}{d(t)} = f'(y)g'(x)h'(t)$$

- Forward Computing
  - Apply an input vector ***x*** to input nodes
  - Computing output vector $\boldsymbol{x}^{(1)}$ on hidden layer

$$x_j^{(1)} = S\left(net_j^{(1)}\right) = S(\sum_i w_{j,i}^{(1,0)} x_i)$$

  - Computing the output vector **y** on output layer

$$y_k = S\left(net_k^{(2)}\right) = S(\sum_j w_{k,j}^{(2,1)} x_j^{(1)})$$

The network is said to be a map from input ***x*** to output **y**

BACKPROPAGATION LEARNING

- Objective of learning:
  - Modify the 2 weight matrices to reduce sum square error

    $\sum_{p=1}^{P}\sum_{k=1}^{K}(l_{p,k})^2$ for the given *P* training samples as much as possible (to zero if possible)

- Idea of BP Learning:
  - Update of weights in $w^{(2,1)}$ (from hidden layer to output layer):

    delta rule as in a single layer net using sum square error
  - Delta rule is not applicable to updating weights in $w^{(1,0)}$ (from input and hidden layer) because we don't know the desired values for hidden nodes
  - **Solution**: Propagating errors at output nodes down to hidden nodes, these computed errors on hidden nodes drives the update of weights in $w^{(1,0)}$ (again by delta rule), thus called error **Back Propagation (BP)** learning

– **How to compute errors on hidden nodes is the key**

BACKPROPAGATION LEARNING

- **Generalized delta rule**:
  - Consider sequential learning mode: for a given sample (x_p, d_p)

$$E = \sum_k (l_{p,k})^2 = \sum_k (d_{p,k} - o_{p,k})^2$$

  - Update weights by gradient descent

    For weight in $w^{(2,1)}$: $\Delta w_{k,j}^{(2,1)} \propto (-\partial E / \partial w_{k,j}^{(2,1)})$

    For weight in $w^{(1,0)}$: $\Delta w_{j,i}^{(1,0)} \propto (-\partial E / \partial w_{j,i}^{(1,0)})$

  - Derivation of update rule for $w^{(2,1)}$:

    since E is a function of $l_k = d_k - y_k$, y_k is a function of $net_k^{(2)}$, and $net_k^{(2)}$ is a function of $w^{(2,1)}$, by chain rule

$$\frac{\partial E}{\partial w_{k,j}^{(2,1)}} = \frac{\partial E}{\partial (d_k - o_k)} \frac{\partial (d_k - o_k)}{\partial net_k^{(2)}} \frac{\partial net_k^{(2)}}{\partial w_{k,j}^{(2,1)}}$$
$$= -2(d_k - o_k) \mathcal{S}'(net_k^{(2)}) x_j^{(1)}.$$

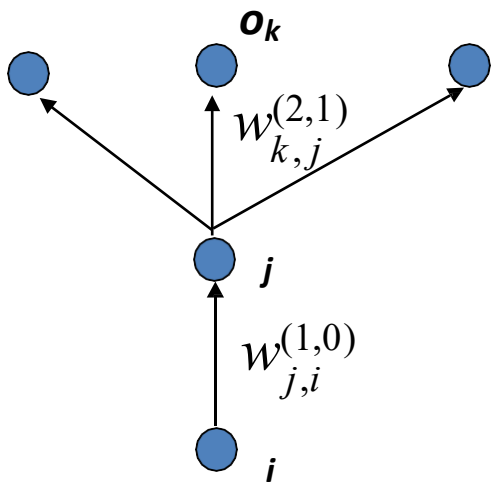
– Derivation of update rule for $w_{j,i}^{(1,0)}$

consider **hidden node *j***:

weight $w_{j,i}^{(1,0)}$ influences $net_j^{(1)}$

it sends $S(net_j^{(1)})$ to all output nodes

$\therefore$ all *K* terms in *E* are functions of $w_{j,i}^{(1,0)}$



$$E = \sum_k (d_k - o_k)^2, \quad o_k = S(net_k^{(2)}), \quad net_k^{(2)} = \sum_j x_j^{(1)} w_{k,j}^{(2,1)},$$

$$x_j^{(1)} = S(net_j^{(1)}), \; net_j^{(1)} = \sum_i x_i w_{j,i}^{(1,0)}$$

by chain rule

$$\frac{\partial E}{\partial w_{j,i}^{(1,0)}} = \sum_{k=1}^{K} \left\{ -2(d_k - o_k) \boxed{\frac{\partial E}{\partial o_k}} \boxed{\frac{\partial S(net_k^{(2)})}{\partial net_k^{(2)}}} \boxed{\frac{\partial net_k^{(2)}}{\partial x_j^{(1)}}} \boxed{\frac{\partial x_j^{(1)}}{\partial net_j^{(1)}}} \boxed{\frac{\partial net_j^{(1)}}{\partial w_{j,i}^{(1)}}} \right\}$$

$$= \sum_{k=1}^{K} \left\{ -2(d_k - o_k) S'\left(net_k^{(2)}\right) w_{k,j}^{(2,1)} S'\left(net_j^{(1)}\right) x_i \right\}$$

OTHER APPLICATIONS

Medical diagnosis

- Input: manifestation (symptoms, lab tests, etc.) Output: possible disease(s)
- Problems:
  - no causal relations can be established
  - hard to determine what should be included as inputs
- Currently focus on more restricted diagnostic tasks
  - e.g., predict prostate cancer or hepatitis B based on standard blood test

Process control

- Input: environmental parameters Output: control parameters
- Learn ill-structured control functions

OTHER APPLICATIONS

Stock market forecasting

- Input: financial factors (CPI, interest rate, etc.) and stock quotes of previous days (weeks)
- Output: forecast of stock prices or stock indices (e.g., S&P 500)
- Training samples: stock market data of past few years

Consumer credit evaluation

- Input: personal financial information (income, debt, payment history, etc.)
- Output: credit rating

SUMMARY OF BP NETS

Architecture

- Multi-layer, feed-forward (full connection between nodes in adjacent layers, no connection within a layer)
- One or more hidden layers with non-linear activation function (most commonly used are sigmoid functions)

BP learning algorithm

- Supervised learning (samples (x_p, d_p))
- Approach: gradient descent to reduce the total error $\Delta w \propto -\partial E / \partial w$
  (why it is also called generalized delta rule)
- Error terms at output nodes
  error terms at hidden nodes (why it is called error BP)
- Ways to speed up the learning process
  - Adding momentum terms
  - Adaptive learning rate (delta-bar-delta)
  - Quickprop
- Generalization (cross-validation test)

Strengths of BP learning

- Great representation power
- Wide practical applicability
- Easy to implement
- Good generalization power

Problems of BP learning

- Learning often takes a long time to converge
- The net is essentially a black box
- Gradient descent approach only guarantees a local minimum error
- Not every function that is representable can be learned
- Generalization is not guaranteed even if the error is reduced to zero
- No well-founded way to assess the quality of BP learning
- Network paralysis may occur (learning is stopped)
- Selection of learning parameters can only be done by trial-and-error
- BP learning is non-incremental (to include new training samples, the network must be re-trained with all old and new samples)

THANK YOU